

Balancing a Ball with a Robotic Arm

Suleyman Soltanov, Brian Van Stratum, Jake-Anthony Vickers

Abstract— We demonstrate the use of modern linear feedback control techniques to make a two-link robot arm move a ball from point to point while balancing. We utilize MATLAB built-in pole placement to design a linear feedback controller with an observer. We utilize a parameter variational study to find a basin of attraction for our control law. Finally, we posit a simple gain schedule including two linearization points to solve a trajectory tracking problem.

I. INTRODUCTION

The goal of this course project is to employ a robotic arm to balance a ball on a level surface and compensate for ball movement induced by human interaction or environmental disturbances [1]. Considering that the surface of the ball is spherical, we know that it forms a higher kinematic couple with other objects, and since it has three degrees of freedom, the process of balancing it is also quite a complex task, which we will attempt to do this throughout the course project. Though the ball balancing problem may appear to be limited in application, one must remember that Marc Raibert has made a career out of making robots dance (something my grandmother does not approve of), and he got to give the keynote speech at IROS in 2022.

In order to explore this problem we model the full non-linear system dynamics using an Euler-Lagrange approach which we explain in section II. Next, in order to make use of linear control techniques we linearize our system. Details of this linearization are provided in section III. We explain our observer and how we controlled the system in section IV-A. More detailed numerical stability analysis is provided in section IV-B. We expand the project to a simple tracking problem in section IV-C and explain our use of a rudimentary gain schedule.

II. SIMULATION (MODELING)

A. full non-linear system modeling

We model the system deriving our dynamical model using the Euler-Lagrange method of determining the Equations of Motion. Since we have three degrees of freedom (DOF). We end up with three coupled dynamic equations given by.

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}_k} - \frac{\partial L}{\partial q_k} = Q_{nc} \quad (1)$$

Where L is the Lagrangian or the difference between the kinetic and potential energies for the system of rigid bodies. Since we have three DOF we have $q = [\theta_1 \ \theta_2 \ s]^T$ and

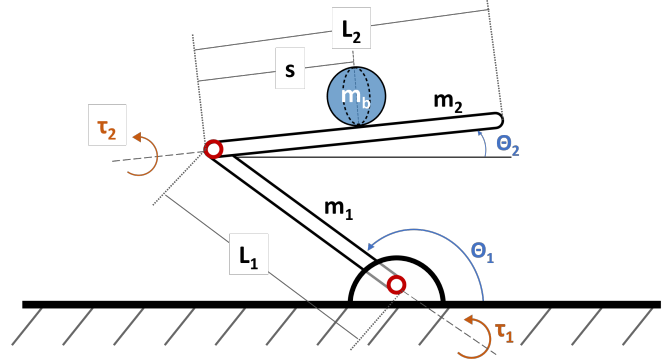


Fig. 1: The two linkage ball balancing robot arm.

$\dot{q} = [\dot{\theta}_1 \ \dot{\theta}_2 \ \dot{s}]^T$ and the index variable $k = 1, 2, 3$. Expanding Equation 1 we have these three coupled dynamic equations:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{\theta}_1} - \frac{\partial L}{\partial \theta_1} = \tau_1 \quad (2)$$

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{\theta}_2} - \frac{\partial L}{\partial \theta_2} = \tau_2 \quad (3)$$

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{s}} - \frac{\partial L}{\partial s} = 0 \quad (4)$$

Equations 2-4 are given in the attached mathematical appendix in section VI

These equations have been recast into the more compact form given below:

$$M(q)\ddot{q} = C(q, \dot{q})\dot{q} + G(q)q + U \quad (5)$$

Where the matrices M and C and G are also provided in the appendix.

In order to validate the generated equations of motion we numerically integrate (simulate) the system using Matlab's built-in variable-step integration tool ODE45. Position states are given in Fig. 2(a) and (b) We verify that if the inputs are zero then the system's sum of kinetic and potential energy remains constant as the system evolves in time. This is required by the first law of thermodynamics and essentially functions as a check that the Lagrangian derivation of the equations of motion is correct. Moreover, we animate for various initial conditions and show that the outputs appear correct intuitively. Note that the ball always falls, and its state is unbounded. It bears mentioning that this is of course the case because the system needs to be controlled to be stable. Animation of the simulation was accomplished using the fixed time stepping method taught in class.

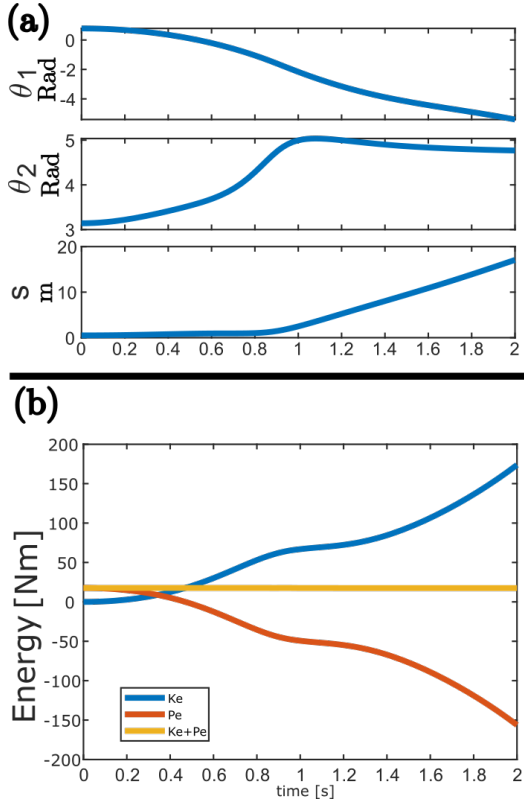


Fig. 2: Validation of Lagrangian Mechanics process.(a) Position state variables as a function of time for initial conditions that are the mirror image of Fig.1. (b) Plots of kinetic and potential energy as a function of time for the same uncontrolled run. Note the sum of kinetic and potential energy remains constant since there are no inputs. Animation of these outputs are included online (*ReportFig2.mp4*) along with various other initial conditions.

B. Motor Saturation

Actuator saturation is a crucial issue to consider during the design process of engineering systems. Dynamic systems usually encounter actuator saturation, which can result in performance degradation and instability in closed-loop systems. Due to the presence of performance limitations and unavoidable uncertainties in saturation problems, the analysis and synthesis of control systems with actuator saturation have become significant topics for any dynamic system.

In order to achieve motor saturation for the given system, we have utilized the tanhx hyperbolic function and have bounded our graph at 17.6 and 10 N*m. By applying control theory concepts, we can ensure that the motor operates within its limits and does not exceed its maximum capacity. The tanhx function is commonly used in control theory as it provides a smooth transition from linear to saturation regions, allowing for precise control of the system Fig.3.

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

where ‘sinh(x)’ and ‘cosh(x)’ are the hyperbolic sine and

Description	Symbol	Value
Link 1		
Moment of inertia	I_1	1.0
Link 2		
Moment of inertia	I_2	1.0
Link one mass	m_1	1.0
Link two mass	m_2	1.0
Ball mass	m_b	1.0
Link one length	L_1	1.0
Link two length	L_2	1.0
Acc. due to gravity	g	10

TABLE I: Parameters used in our study. All units are standard SI.

cosine functions respectively.

The ‘tanh(x)’ function has a range between -1 and 1, and is symmetric about the origin. It is commonly used to model the saturation behavior of motors, where the output torque or power of the motor approaches a maximum value as the input current increases.

Furthermore, by setting the bounds at 17.6 and 10 N*m, we can prevent the motor from operating beyond its safe operating range. This is crucial in ensuring the longevity of the motor and preventing any potential damage or failure.

III. LINEARIZATION

Linearization is a crucial step in the control theory of dynamic systems. It involves approximating a nonlinear system by a linear one around an equilibrium point or operating point. In the context of designing a robotic arm that can balance a ball on a level surface, linearization is an essential step to achieve stability and control.

$$\begin{aligned} \dot{\hat{x}}(t) = f(x^*, u^*) &+ \left. \frac{\partial f(x(t), u(t))}{\partial x(t)} \right|_{x^*, u^*} (x(t) - x^*) \\ &+ \left. \frac{\partial f(x(t), u(t))}{\partial u(t)} \right|_{x^*, u^*} (u(t) - u^*) \end{aligned}$$

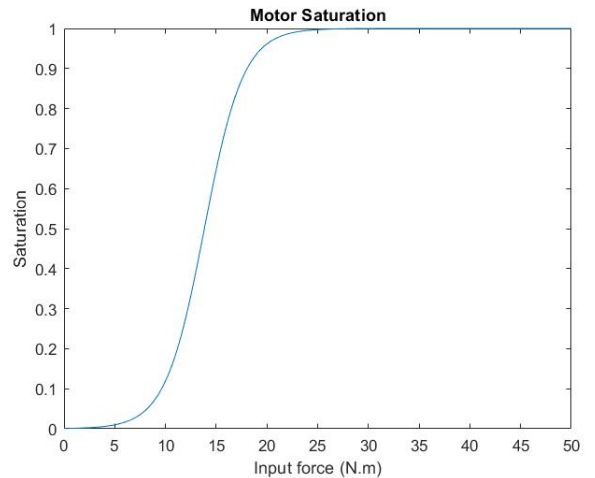


Fig. 3: Actuator Saturation for the given system.

This equation represents a linearized approximation of the dynamics of a system around an equilibrium point (x^*, u^*) . The first term $f(x^*, u^*)$ represents the steady-state behavior of the system, while the other two terms represent the effects of deviations from the equilibrium point. The second term is proportional to the deviation of the state variables $x(t)$ from their equilibrium values x^* , weighted by the sensitivity of the system to these deviations, as captured by the partial derivative $\frac{\partial f(x(t), u(t))}{\partial x(t)}$. The third term is proportional to the deviation of the input variables $u(t)$ from their equilibrium values u^* , weighted by the sensitivity of the system to these deviations, as captured by the partial derivative $\frac{\partial f(x(t), u(t))}{\partial u(t)}$.

To begin with, the design process involved deriving an equation of motion for the given system. This equation was nonlinear due to the presence of the ball's position and velocity. However, to make it amenable to control theory techniques, we needed to linearize it around a fixed point.

$$x^* = \begin{bmatrix} \frac{\pi}{4} & 0 & 0 & 0 & 0.5 & 0 \end{bmatrix}^T; u^* = \begin{bmatrix} 17.677 & 10 \end{bmatrix}.$$

We used the linearization method from our control theory course to approximate the nonlinear system by a linear one. Specifically, we computed the Jacobian matrix of the nonlinear system evaluated at the fixed point. This matrix gave us information about the local behavior of the system near the fixed point.

The Jacobian matrix is a matrix of partial derivatives that describes the rate of change of a set of output variables with respect to a set of input variables. In this case, we want to find the Jacobian matrix for the functions $f=[f_1, f_2, f_3, f_4, f_5, f_6]$ with respect to the variables $x=[\theta_1, \dot{\theta}_1, \theta_2, \dot{\theta}_2, s, \dot{s}]$. The Jacobian matrix J is defined as:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial \theta_1} & \frac{\partial f_1}{\partial \dot{\theta}_1} & \frac{\partial f_1}{\partial \theta_2} & \frac{\partial f_1}{\partial \dot{\theta}_2} & \frac{\partial f_1}{\partial s} & \frac{\partial f_1}{\partial \dot{s}} \\ \frac{\partial f_2}{\partial \theta_1} & \frac{\partial f_2}{\partial \dot{\theta}_1} & \frac{\partial f_2}{\partial \theta_2} & \frac{\partial f_2}{\partial \dot{\theta}_2} & \frac{\partial f_2}{\partial s} & \frac{\partial f_2}{\partial \dot{s}} \\ \frac{\partial f_3}{\partial \theta_1} & \frac{\partial f_3}{\partial \dot{\theta}_1} & \frac{\partial f_3}{\partial \theta_2} & \frac{\partial f_3}{\partial \dot{\theta}_2} & \frac{\partial f_3}{\partial s} & \frac{\partial f_3}{\partial \dot{s}} \\ \frac{\partial f_4}{\partial \theta_1} & \frac{\partial f_4}{\partial \dot{\theta}_1} & \frac{\partial f_4}{\partial \theta_2} & \frac{\partial f_4}{\partial \dot{\theta}_2} & \frac{\partial f_4}{\partial s} & \frac{\partial f_4}{\partial \dot{s}} \\ \frac{\partial f_5}{\partial \theta_1} & \frac{\partial f_5}{\partial \dot{\theta}_1} & \frac{\partial f_5}{\partial \theta_2} & \frac{\partial f_5}{\partial \dot{\theta}_2} & \frac{\partial f_5}{\partial s} & \frac{\partial f_5}{\partial \dot{s}} \\ \frac{\partial f_6}{\partial \theta_1} & \frac{\partial f_6}{\partial \dot{\theta}_1} & \frac{\partial f_6}{\partial \theta_2} & \frac{\partial f_6}{\partial \dot{\theta}_2} & \frac{\partial f_6}{\partial s} & \frac{\partial f_6}{\partial \dot{s}} \end{bmatrix}$$

Where $\dot{x}$ represents the time derivative of x . To find the individual elements of the Jacobian matrix, we need to take the partial derivative of each function with respect to each variable.

Using this Jacobian matrix, we computed the A and B matrices for the linearized system. The A matrix describes how the state variables evolve over time, while the B matrix describes how control inputs affect this evolution. These matrices are crucial for designing feedback controllers that can stabilize and control the system.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 5.1722 & 0 & -12.0687 & 0 & 1.3793 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 17.677 & 0 \\ 0 & 10 \\ 0 & 0 \end{bmatrix}$$

In the context of control theory, it is crucial to determine the controllability and stability of a given dynamic system. In this regard, I have identified A and B matrices that define the linearization of the system. By calculating the controllability matrix and taking the rank of A and B matrices, we have established that the system is fully controllable. However, after computing the eigenvalues of A matrix, we have discovered that the system is not stable. Therefore, it is imperative to design a control strategy that can stabilize the system and ensure its optimal performance.

We have conducted Stability and Controllability analysis for our system using Matlab as follows:

```
Eigenvalues = eig(A)
if eig(A) < 0
disp('The system is stable.')
else
disp('The system is NOT stable.')
end
% The system is NOT stable.

Cont = ctrb(A,B)
[rowsA, columnsA] = size(A);
% Get # rows of A matrix
if rank(ctrb(A,B)) == rowsA
disp('\textbf{Fully controllable.}')
else
disp('NOT fully controllable.')
end
% The system is fully controllable.
```

IV. CONTROL SYSTEM DESIGN

This linear system was controlled with a gain matrix computed with MATLAB's place command. Moreover, an observer was designed based on a C matrix that limits the sensed outputs only to the position states θ_1 θ_2 s and Stability was tested about the operating point depicted in figure 1. It was found from trial and error testing that a somewhat narrow range of poles effectively balances the ball. If poles are too slow then gravity can pull the ball away before the controller can respond while the dynamics are sufficiently close to the operating point. Also, if the controller's gains are too fast the robot will respond too quickly, causing the ball to fling off the end of the arm. Section IV-A explains the pole placement controller. Section IV-B shows the result of a numerical study on the controller's effectiveness analyzing its performance as a function of poles and disturbances. Finally, section IV-C explains how we tackle the trajectory tracking problem.

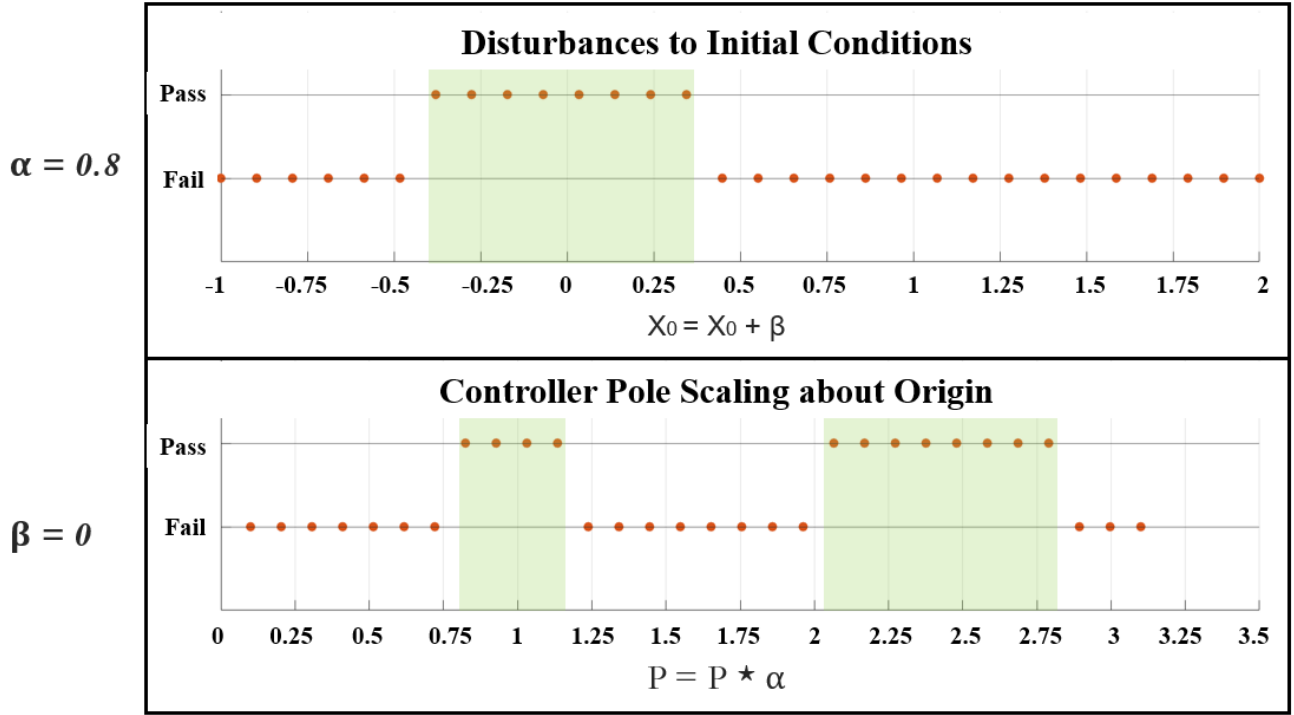


Fig. 4: Sweep of various (a) Disturbances to Initial Conditions and (b) Pole Scaling Multiplier

A. Control System Description

Work from the previous section provided an LTI system approximation of the form

$$\begin{aligned}\dot{\mathbf{X}} &= \mathbf{A} \mathbf{X} + \mathbf{B} \mathbf{U} \\ \mathbf{Y} &= \mathbf{C} \mathbf{X} + \mathbf{D} \mathbf{U}\end{aligned}$$

that allowed us to make use of the many tools available in the MATLAB programming environment. We chose to design of controller of the form

$$\mathbf{U} = -\mathbf{K} \mathbf{X}$$

we used the MATLAB “PLACE” command to find a gain matrix $\mathbf{K}$ with a error decay rate specified by 6 system poles that we get to choose. We found that placing the poles $\mathbf{P}$ at

$$\mathbf{P} = [-3 \quad -4 \quad -5 \quad -6 \quad -7 \quad -8]$$

was generally acceptable for many of the robot configurations. It was noted however that deviating from these poles by too much would cause the robot to be unable to regulate the ball position.

In order to make our project more interesting we chose to limit the sensed output only to the position states. This implies a $\mathbf{C}$ matrix of the form

$$\mathbf{C} = \begin{bmatrix} 1 & & & & & \\ & 1 & & & & \\ & & 1 & & & \\ & & & 0 & & \\ & & & & 0 & \\ & & & & & 0 \end{bmatrix}$$

Along with this we designed an observer again using the MATLAB PLACE command. Observer gain poles $\mathbf{L}$ were chosen to be ten times faster than the controller gains therefore we have.

$$\mathbf{L} = 10 \mathbf{P}$$

Introducing the observer allowed us to consider the effect of flaws in the state estimate $\hat{\mathbf{X}}$. Generally it was discovered that errors in the state estimate was about as costly to the controller as disturbances. However we did not make a detailed study of this. In the following section we do make a detailed study of the effect of scaling the pole location and the effect of state disturbance as understood by varying the perturbation of the full state $\mathbf{X}_0$ away from $\mathbf{X}_*$

B. Analysis of working Poles

From the observer, we discovered that the working poles of the system were located at two extrema, we decided to analyze the stability of the working poles of the system. First, we defined two new variables, α and β , to $\mathbf{X}_0$, and the poles matrix, $\mathbf{P}$. Then a sweep of the system dynamics using different disturbances added to the initial conditions was performed, where β represents the magnitude of the initial disturbance. We ranged β from $-1 \leq \beta \leq 2$. The final values of the state matrix were compared to the expected final values and if the difference was below $1e-1$ and then a “pass” or “fail” was assigned to the value. The top plot of fig. 4 shows a plot of the sweeps with the “passing” values highlighted in green.

This provided a general range of how far from the linearization point we could move before the system goes

unstable.

The next step was to analyze the pole multiplier, α . We ranged this value from $0.1 \leq \alpha \leq 3.1$, and just as with β , the final value was compared to a tolerance and if it converged, the system “passed.” We see in the lower plot of fig 4, that just as in the observer, there are two ranges for α where the system converges back to its fixed point. This indicates that the poles can be “too slow”, where the arm cannot react fast enough to catch the ball. Additionally, the poles can be “too fast” where the arm just flings the ball into the air.

C. Global reference frame tracking

In order to start to understand the challenge of something more interesting than state regulation. We proposed the problem of moving the ball from one position to another. In order to command the ball position in the global reference frame as a function of time we first specify the global reference frame position of the ball and then compute the inverse kinematics to find the desired tracking state. As a simple test case in particular, we define the ball position in the global reference frame as follows

$$\begin{aligned} y_{ball}(t) &= \dot{y}t + y_0 \\ x_{ball} &= x_0 \end{aligned}$$

Where the user chooses $\dot{y}$, y_0 and x_0 and must be in the arms workspace, computing the inverse kinematics on these global reference kinematics gives these results.

$$\theta_1(t) = \arcsin\left(\frac{\dot{y}t + y_0}{L_1}\right) \quad (6)$$

$$\theta_2 = \pi \quad (7)$$

$$s(t) = \sqrt{L_1^2 - (\dot{y}t + y_0)^2} - x_0 \quad (8)$$

Ideally, derivatives should also be computed, but we did not do that. We chose to use an admittedly naive controller compared to the one found in the Schmidt notes, taking the form

$$\mathbf{U} = -\mathbf{K} \left[(\hat{\mathbf{X}} - \mathbf{X}_*) - (\mathbf{X}_d - \mathbf{X}_*) \right] + \mathbf{U}_* \quad (9)$$

which becomes a familiar formula:

$$\mathbf{U} = \mathbf{K}(\mathbf{X}_d - \hat{\mathbf{X}}) + \mathbf{U}_* \quad (10)$$

Where $\mathbf{X}_d$ is the desired trajectory from the inverse kinematics and gain matrix $\mathbf{K}$ was found as described in section IV-A for the $\mathbf{X}_*$ and $\mathbf{U}_*$ at time $t = 0$.

It was discovered that a single gain matrix $\mathbf{K}$ and $\mathbf{U}_*$ were not stable for this trajectory tracking problem. We addressed this problem by proposing a crude gain schedule and a new controller of this form

$$\mathbf{U}(t) = \begin{cases} \mathbf{K}(\mathbf{X}_d - \hat{\mathbf{X}}) + \mathbf{U}_* & \text{if } t < 4 \\ \mathbf{K}_f(\mathbf{X}_d - \hat{\mathbf{X}}) + \mathbf{U}_{*f} & \text{if } t \geq 4 \end{cases} \quad (11)$$

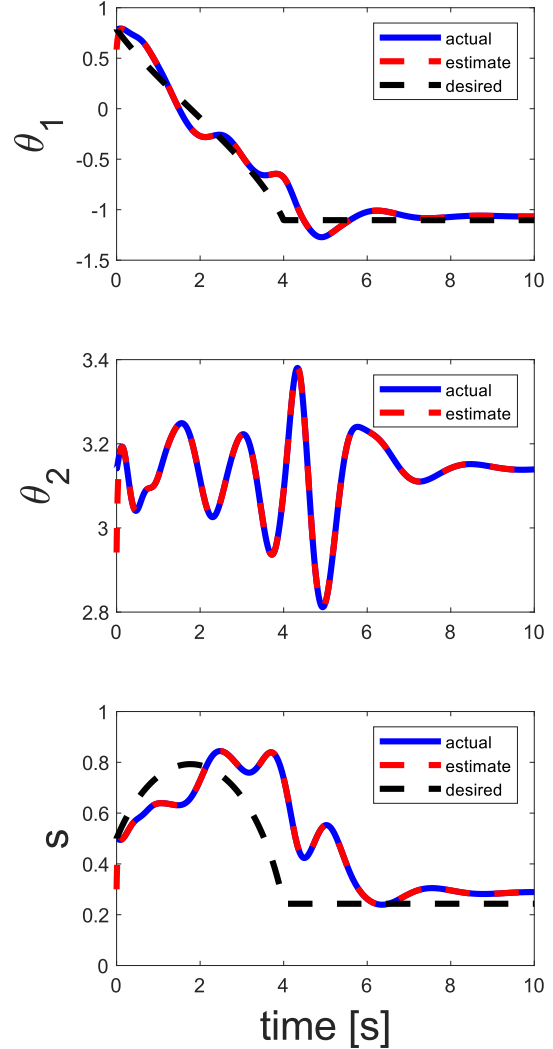


Fig. 5: Robot Arm performance on the trajectory tracking problem. The dashed black line represents the desired trajectory found from the inverse kinematics given in equations 6 - 8. The Red dashed line represents the observer estimate and can be seen to converge very early. The blue line is the actual system response. Video is included.

Where $\mathbf{K}_f$ and $\mathbf{U}_{*f}$ are distinct gain matrices and inputs computed from a second linearization about a new operating point $\mathbf{X}_{*f} = \mathbf{X}_d|_{t=4}$. The performance of this new gain schedule-based approach is shown in figure 5. As can be seen from the θ_2 and s plots, the system begins to go unstable close to $t = 4$, but then seemingly “magically” stabilizes with the application of $\mathbf{K}_f$ and $\mathbf{U}_{*f}$. We note that better performance was obtained in this trajectory tracking case, by making $\mathbf{C}$ an identity matrix. In other words, the controller performed better when the observer had access to the full state. We include trajectory tracking videos for both the full

and half state feedback.

V. ACKNOWLEDGMENTS

Brian Van Stratum derived the equations of motion, and designed the controller and observer, including the trajectory tracking gain schedule controller.

Jake-Anthony Vickers created the animation engine for the system and performed a stability sweep of working poles.

Suleyman Soltanov utilized the equation of motion to identify the fixed point for the given system. Following this, he linearized the system around that operating point. Furthermore, he formulated the motor saturation component by utilizing the hyperbolic tangent function, also known as $\tanh(x)$.

REFERENCES

- [1] A. Kassem, H. Haddad, and C. Albitar, "Commpparison between different methods of control of ball and plate system with 6dof stewart platform," *IFAC-PapersOnLine*, vol. 48, no. 11, pp. 47–52, 2015.

VI. MATHEMATICAL APPENDIX

$$I_1 \ddot{\theta}_1 + \frac{L_1^2 m_1 \dot{\theta}_1}{4} + L_1^2 m_2 \ddot{\theta}_1 + L_1^2 m_b \ddot{\theta}_1 + \frac{L_1 g m_1 \cos(\theta_1)}{2} + L_1 g m_2 \cos(\theta_1) + L_1 g m_b \cos(\theta_1) - L_1 m_b \ddot{s} \sin(\theta_1 - \theta_2) + L_1 m_b s \ddot{\theta}_2 \cos(\theta_1 - \theta_2) + 2 L_1 m_b \dot{s} \dot{\theta}_2 \cos(\theta_1 - \theta_2) + \frac{L_1 L_2 m_2 \dot{\theta}_2^2 \sin(\theta_1 - \theta_2)}{2} + L_1 m_b s \dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + \frac{L_1 L_2 m_2 \ddot{\theta}_2 \cos(\theta_1 - \theta_2)}{2} \quad (12)$$

$$I_2 \ddot{\theta}_2 + m_b s^2 \ddot{\theta}_2 + \frac{L_2^2 m_2 \ddot{\theta}_2}{4} + 2 m_b s \dot{s} \dot{\theta}_2 + \frac{L_2 g m_2 \cos(\theta_2)}{2} + g m_b s \cos(\theta_2) + L_1 m_b s \ddot{\theta}_1 \cos(\theta_1 - \theta_2) - \frac{L_1 L_2 m_2 \dot{\theta}_1^2 \sin(\theta_1 - \theta_2)}{2} - L_1 m_b s \dot{\theta}_1^2 \sin(\theta_1 - \theta_2) + \frac{L_1 L_2 m_2 \ddot{\theta}_1 \cos(\theta_1 - \theta_2)}{2} \quad (13)$$

$$-m_b \left(L_1 \cos(\theta_1 - \theta_2) \dot{\theta}_1^2 + s \dot{\theta}_2^2 - \ddot{s} - g \sin(\theta_2) + L_1 \ddot{\theta}_1 \sin(\theta_1 - \theta_2) \right) \quad (14)$$

extracting the matrices...

$$M = \begin{pmatrix} I_1 + \frac{L_1^2 m_1}{4} + L_1^2 m_2 + L_1^2 m_b & \frac{L_1 \cos(\theta_1 - \theta_2)}{2} (L_2 m_2 + 2 m_b s) & -L_1 m_b \sin(\theta_1 - \theta_2) \\ \frac{L_1 \cos(\theta_1 - \theta_2)}{2} (L_2 m_2 + 2 m_b s) & \frac{m_2 L_2^2}{4} + m_b s^2 + I_2 & 0 \\ -L_1 m_b \sin(\theta_1 - \theta_2) & 0 & m_b \end{pmatrix} \quad (15)$$

$$C = \begin{pmatrix} 0 & \frac{L_1 \sin(\theta_1 - \theta_2)}{2} (L_2 m_2 + 2 m_b s) & 2 L_2 m_b \frac{\dot{\theta}_2}{s} \cos(\theta_1 - \theta_2) \\ -\frac{L_1 \sin(\theta_1 - \theta_2)}{2} (L_2 m_2 + 2 m_b s) & 0 & 2 m_b \frac{\dot{\theta}_2}{s} \\ -L_1 m_b \cos(\theta_1 - \theta_2) & -m_b s & 0 \end{pmatrix} \quad (16)$$

$$G = \begin{pmatrix} \frac{L_1 g \cos(\theta_1)}{2} (m_1 + 2 m_2 + 2 m_b) \\ \frac{g \cos(\theta_2)}{2} (L_2 m_2 + 2 m_b s) \\ g m_b \sin(\theta_2) \end{pmatrix} \quad (17)$$